

Linux Command Line: 30-Day Practitioner's Curriculum

Practical command-line mastery, day by day — beginner to production-ready.

Format: Debian/Ubuntu-family shell (bash), with notes for embedded Linux targets (Raspberry Pi, BeagleBone) where relevant. Every day: concept → annotated commands → key takeaways. No filler theory.

DAY 1 — Shell Fundamentals & Filesystem Navigation

Concept

The shell is a REPL over the OS. Everything is a process talking to the kernel via syscalls; commands are just programs on `$PATH`. The filesystem is a single tree rooted at `/` — there are no drive letters, only mount points.

Core commands

```
pwd                # print working directory (absolute path)
ls                 # list directory contents
ls -la            # long format, include dotfiles (. and ..)
ls -lh            # human-readable sizes (K/M/G)
ls -lt            # sort by modification time, newest first
ls -R             # recursive listing

cd /var/log        # absolute path
cd ../../          # relative: up two levels
cd ~               # home directory
cd -               # previous directory (toggle)
cd                # no arg = home

file somefile      # identify file type by content (not extension)
stat somefile      # inode, size, timestamps, permissions in detail
```

Path mechanics

- **Absolute path:** starts with `/`, unambiguous from anywhere (`/etc/mosquitto/mosquitto.conf`)
- **Relative path:** resolved from CWD (`../config/mosquitto.conf`)
- `.` = current dir, `..` = parent dir, `~` = `$HOME`, `-` = previous dir (only for `cd`)
- Tab completion is not optional — use it constantly; it prevents typos in paths and confirms existence

Filesystem Hierarchy Standard (practical subset)

```
/etc      configuration files (mosquitto.conf, systemd units, fstab)
/var/log  logs
/var/lib  persistent application state (databases, service data)
/usr/bin  most installed program binaries
/usr/local/bin manually installed / custom binaries
/home     user directories
/tmp      volatile scratch space, cleared on reboot
/dev      device files (serial ports, block devices – relevant later)
/proc     virtual filesystem exposing kernel/process state (not real files on disk)
/sys      virtual filesystem exposing kernel objects/drivers (GPIO, etc. on embedded boards)
```

Getting help without leaving the terminal

```
man ls            # full manual page
ls --help         # quick usage summary
whatis ls         # one-line description
apropos serial    # search man page descriptions by keyword
type cd           # is this a builtin, alias, or binary?
which python3     # resolve full path of an executable on $PATH
```

Key takeaways

- Everything is a path; know the difference between absolute and relative.

- `/proc` and `/sys` are not "real" files — they're the kernel talking to you. You'll use these constantly on embedded targets.
 - `man + --help` + tab completion replace 90% of "how do I..." searches.
-

DAY 2 — File Operations & Globbing

Concept

File manipulation commands are simple, composable, and destructive by default — there is no trash can. `rm` does not ask twice.

Core commands

```
touch newfile.txt           # create empty file, or update mtime if it exists
mkdir configs               # create directory
mkdir -p a/b/c              # create nested path in one shot, no error if exists

cp source.txt dest.txt      # copy file
cp -r sourcedir/ destdir/   # copy directory recursively
cp -a sourcedir/ destdir/   # archive mode: preserves perms, timestamps, symlinks

mv old.txt new.txt          # rename (same as move, on same filesystem it's instant – no data copy)
mv file.txt /var/tmp/       # move into directory

rm file.txt                 # delete file – NOT recoverable
rm -r directory/           # recursive delete
rm -rf directory/          # recursive + force (no prompts, ignores missing) – the most dangerous command
rmdir emptydir             # remove only if empty (safer than rm -r for dirs)
```

Globbing (pattern expansion done BY THE SHELL, before the command sees it)

```
ls *.log                   # all files ending in .log
ls sensor_*.csv            # prefix match
ls file?.txt               # ? = exactly one character
ls [abc]*.txt              # character class: starts with a, b, or c
ls [0-9]*.dat              # numeric range
ls **/*.py                 # recursive glob (requires: shopt -s globstar)
```

Critical mental model: the shell expands `*.log` into a literal list of filenames **before** `ls` (or `rm`, or anything) ever runs. `rm *.log` deletes files matching that pattern in the current directory — nothing more, nothing magic.

Safety habits that matter in practice

```
ls *.log                   # ALWAYS preview the glob with ls before using it with rm
rm -i *.log                # -i = interactive, confirm each deletion
cp -i src dst              # -i also works for cp/mv, prevents silent overwrite
```

Symbolic and hard links

```
ln -s /etc/mosquitto/mosquitto.conf ~/mosquitto.conf # symlink: pointer to a path
ln original.txt hardlink.txt                          # hard link: second name for same inode
ls -li                                                 # show inode numbers to prove it
```

- Symlink: breaks if target moves/deletes; can cross filesystems; shows as `l` in `ls -li`.
- Hard link: same inode, same data; cannot cross filesystems; deleting one name doesn't delete data until all links are gone.

Key takeaways

- The shell expands globs, not the command — this explains a lot of "weird" behavior.
- Always `ls` a glob pattern before you `rm` it.
- `mv` on the same filesystem is a metadata rename (instant, no I/O); across filesystems it's copy+delete.

DAY 3 — Viewing & Inspecting File Contents

Concept

You will spend more time reading files (especially logs) than editing them. Master the viewers before the editors.

Core commands

```
cat file.txt # dump entire file to stdout – fine for small files
cat -n file.txt # with line numbers
cat file1 file2 > combined.txt # concatenate (this is literally what "cat" means)

less file.txt # paginated viewer – THE tool for anything non-trivial
# inside less:
# space/b page down/up
# /pattern search forward, n = next match, N = previous
# ?pattern search backward
# g / G go to top / bottom
# q quit

more file.txt # older, more limited pager (less is strictly better; "less is more")

head file.txt # first 10 lines
head -n 50 file.txt # first 50 lines
head -c 100 file.txt # first 100 bytes

tail file.txt # last 10 lines
tail -n 100 file.txt # last 100 lines
tail -f /var/log/syslog # follow: stream new lines as they're appended – essential for live log monitoring
tail -F /var/log/app.log # like -f but re-attaches if the file is rotated/recreated (use this in production)

wc file.txt # lines, words, bytes
wc -l file.txt # just line count (most common use)
```

Practical patterns you'll use constantly

```
tail -F /var/log/mosquitto/mosquitto.log # watch MQTT broker logs live
journalctl -u mqtt_monitor -f # (preview – covered Day 12) follow a systemd service's logs
head -n 20 sensor_data.csv | column -t -s, # peek at a CSV, aligned into columns
watch -n 2 'ls -la /dev/ttyUSB*' # re-run a command every 2s – useful for watching device nodes
```

Binary/hex inspection

```
xxd file.bin | head # hex + ASCII dump – for inspecting binary protocol data
od -c file.bin | head # octal dump, character interpretation
strings binaryfile # extract printable strings from a binary – quick triage tool
```

Key takeaways

- **less** for anything you're reading interactively; **cat** only for short files or piping.
 - **tail -F** (capital F), not **-f**, when watching a log that might rotate.
 - **strings** + **xxd** are your first move when inspecting unknown binary data (e.g., a serial dump).
-

DAY 4 — Permissions & Ownership

Concept

Every file has an owner, a group, and a permission triplet (owner/group/other) × (read/write/execute). This is the single most common source of "permission denied" confusion, and it's fully deterministic once you internalize the model.

Reading `ls -l` output

```
-rwxr-xr-- 1 george staff 4096 Jul 20 10:00 monitor.sh
```

- `-` = file type (`-` regular, `d` directory, `l` symlink, `c` char device, `b` block device)
- `rwx` = owner permissions (read, write, execute)
- `r-x` = group permissions
- `r--` = other (everyone else) permissions
- `george` = owning user, `staff` = owning group

Changing permissions

```
chmod u+x script.sh      # add execute for owner (user)
chmod g-w file.txt       # remove write for group
chmod o=r file.txt       # set "other" to read-only, exactly
chmod +x script.sh       # add execute for everyone (shorthand)
chmod 755 script.sh      # numeric: rwxr-xr-x (owner=7, group=5, other=5)
chmod 644 config.txt     # numeric: rw-r--r-- (typical for config/data files)
chmod -R 755 scripts/    # recursive
```

Numeric permission mental model

Each digit = sum of: read=4, write=2, execute=1.

- 7 = rwx (4+2+1)
- 6 = rw- (4+2)
- 5 = r-x (4+1)
- 4 = r--
- 0 = ---

Common patterns: `755` executables/dirs you own, `644` regular data files, `600` private keys/secrets, `700` private directories.

Ownership

```
chown george file.txt      # change owner
chown george:staff file.txt # change owner AND group
chown -R george:staff project/ # recursive
chgrp staff file.txt       # change group only
```

Directory execute bit — the thing that confuses everyone

On a **directory**, the execute bit doesn't mean "run it" — it means "can traverse into it" (needed to `cd` into it or access files inside by path, even if you can't `ls` it). Read bit on a directory means "can list its contents." You need **both** `r` and `x` on a directory to browse it normally.

umask — default permission mask

```
umask                # show current mask, e.g. 0022
umask 022            # new files get 666-022=644, new dirs get 777-022=755
```

Special bits (used less often, but you'll meet them)

```
chmod u+s /usr/bin/some_binary    # setuid: runs as the file's owner, not the invoking user (e.g. passwd)
chmod g+s shared_dir/             # setgid on dir: new files inherit the dir's group
chmod +t /tmp                     # sticky bit: only the file's owner can delete it, even with dir write p
```

Key takeaways

- **rwX** for owner/group/other, translate to octal (4/2/1) fluently — you'll type **chmod 755** and **644** daily.
 - Directory execute bit = traversal, not "run." This trips up everyone once.
 - **600** for anything sensitive (SSH keys, credentials files, **.env**).
-

DAY 5 — Users, Groups & Privilege Escalation

Concept

Linux is multi-user by design. Every process runs as a UID; permission checks are UID/GID-based, not username-based (usernames are just a lookup convenience).

Key files

```
cat /etc/passwd      # user accounts: username:x:UID:GID:comment:home:shell
cat /etc/group        # group definitions: groupname:x:GID:member1,member2
cat /etc/shadow       # hashed passwords (root-readable only)
```

Example `/etc/passwd` line: `george:x:1000:1000:George:/home/george:/bin/bash` — fields are username, password placeholder, UID, GID, GECOS comment, home dir, login shell.

User/group management

```
sudo useradd -m -s /bin/bash monitoruser  # create user, -m makes home dir, -s sets shell
sudo passwd monitoruser                   # set/change password
sudo userdel -r monitoruser                # delete user + home dir

sudo groupadd iot                          # create group
sudo usermod -aG iot george                # add george to group iot (-a = append, don't forget -a or you
groups george                             # show george's group memberships
id                                          # show current user's UID, GID, and all group memberships
id george                                 # same, for another user
```

Critical gotcha: `usermod -G iot george` (no `-a`) **replaces** all of george's supplementary groups with just `iot`. Always use `-aG`.

Switching users / gaining privilege

```
su - otheruser      # switch user, load their full environment (the dash matters)
su otheruser        # switch user, keep current shell environment (usually not what you want)

sudo command        # run a single command as root (or another user, per /etc/sudoers)
sudo -u monitoruser command  # run as a specific non-root user
sudo -i             # get a root login shell
sudo !!             # re-run the previous command with sudo (very common pattern after "permission denied")

visudo              # SAFELY edit /etc/sudoers (validates syntax before saving – never edit it with a
```

Why groups matter for device access (embedded-relevant)

On most distros, `/dev/ttyUSB0` and `/dev/ttyACM0` (USB-serial adapters) are owned by group `dialout`. Instead of running your monitor as root:

```
sudo usermod -aG dialout george
# log out and back in for group membership to take effect
groups                          # confirm 'dialout' is now listed
```

This is the correct, minimal-privilege way to get serial port access instead of `sudo`-ing everything.

Key takeaways

- UID is the real identity; usernames are cosmetic. `id` is your fastest diagnostic command.
- `usermod -aG`, always with `-a`, or you silently destroy other group memberships.

- Prefer adding your user to the right group (`dialout`, `gpio`, `i2c` on embedded boards) over running services as root.
-

DAY 6 — Text Processing I: grep & Regular Expressions

Concept

`grep` searches text by pattern. It is the single highest-leverage command line tool for log analysis and code search. Regex fluency here pays off across `grep`, `sed`, `awk`, and beyond.

Core usage

```
grep "error" app.log          # lines containing "error" (case-sensitive, literal substring)
grep -i "error" app.log       # case-insensitive
grep -v "debug" app.log       # invert match: lines NOT containing "debug"
grep -n "error" app.log       # show line numbers
grep -c "error" app.log       # count matching lines (not occurrences)
grep -r "TODO" src/           # recursive search through a directory tree
grep -l "mqtt_connect" *.py   # list only filenames that match (not the lines)
grep -w "conn" app.log        # whole word only (not substring of another word)
grep -A 3 "Traceback" app.log  # show 3 lines After the match (context)
grep -B 3 "Traceback" app.log  # 3 lines Before
grep -C 3 "Traceback" app.log  # 3 lines of Context on both sides
grep -E "error|warning|critical" app.log # extended regex, alternation
grep -o "sensor_[0-9]\+" app.log # print only the matched portion, not the whole line
```

Regex essentials (BRE/ERE — POSIX, what grep/sed use)

.	any single character
*	zero or more of the preceding atom
+	one or more (requires -E or \+ in basic regex)
?	zero or one (requires -E or \? in basic regex)
^	start of line
\$	end of line
[abc]	character class: a, b, or c
[^abc]	negated class: not a, b, or c
[0-9]	range
\d	NOT in POSIX grep — use [0-9] instead (or use grep -P for Perl regex)
()	grouping (requires -E or escaped \(\) in basic regex)
	alternation (requires -E)
{n,m}	repetition count (requires -E or escaped)
\	escape a special character

Practical examples against real log lines

```
grep -E "^\[ERROR\]" app.log          # lines starting with literal [ERROR]
grep -E "temperature: [0-9]+\.[0-9]+" data.log # match a decimal number pattern
grep -P "\d{4}-\d{2}-\d{2}" app.log     # ISO date, using Perl-compatible regex (-P)
grep -E "192\.168\.[0-9]+\.[0-9]+" app.log # rough IP match (escaping the literal dots)
```

Key takeaways

- `grep -E` (extended regex) is what you want 90% of the time — don't fight basic regex's escaping rules.
 - `-i -n -v -c -r` are the five flags you'll type without thinking within a week.
 - `grep -P` gives you Perl regex (`\d`, `\w`, lookaheads) when POSIX classes feel too limited.
-

DAY 7 — Text Processing II: sed

Concept

sed (stream editor) transforms text line-by-line as it streams through — non-interactive find/replace, deletion, and insertion, ideal for scripted, repeatable edits (config templating, log scrubbing, batch renames of content).

Core usage

```
sed 's/foo/bar/' file.txt          # replace FIRST occurrence of foo->bar on each line, print to stdout
sed 's/foo/bar/g' file.txt        # replace ALL occurrences on each line (global)
sed -i 's/foo/bar/g' file.txt     # -i = edit the file IN PLACE (no output to stdout)
sed -i.bak 's/foo/bar/g' file.txt # in-place, but keep a .bak backup first (safer habit)

sed -n '5,10p' file.txt           # print only lines 5-10 (-n suppresses default auto-print, p prints)
sed '5,10d' file.txt              # delete lines 5-10
sed '/^#/d' file.txt              # delete all comment lines (lines starting with #)
sed '/^$/d' file.txt              # delete all blank lines

sed 's/ERROR/CRITICAL/gi' file.txt # case-insensitive global replace
sed 's/[0-9]\+/NUM/g' file.txt     # replace all number sequences with "NUM"

sed -n '/START/,/END/p' file.txt   # print everything between two markers, inclusive
```

Practical: editing config files safely

```
sed -i 's/^port .*/port 8883/' /etc/mosquitto/mosquitto.conf # change the MQTT broker port
sed -i 's/#listener/listener/' /etc/mosquitto/mosquitto.conf # uncomment a directive
sed -n '/\[mqtt\]/,/\[/p' config.ini                        # extract one section of an INI file
```

Multiple edits at once

```
sed -e 's/foo/bar/' -e 's/baz/qux/' file.txt # chain multiple substitutions
sed 's/foo/bar/; s/baz/qux/' file.txt        # semicolon-separated, same effect
```

Using a different delimiter (when your pattern contains /)

```
sed 's#/var/log#/mnt/logs#g' file.txt # use # instead of / when replacing paths
```

Key takeaways

- **sed -i.bak** for anything real — the backup costs nothing and saves you once.
 - **s/pattern/replacement/g** is the 90% use case; **-n '/start/,/end/p'** is the second most useful.
 - Change the delimiter (**s#...#...#**) whenever your text contains slashes, to avoid escaping hell.
-

DAY 8 — Text Processing III: awk

Concept

awk is a full pattern-scanning and field-processing language — where **grep** finds lines and **sed** transforms text, **awk** treats each line as a record split into fields, ideal for columnar/structured data (CSV, log lines, **ps/df** output).

Core mental model

```
awk '{ print $1 }' file.txt           # print field 1 of every line (default field separator = whitespace)
awk '{ print $1, $3 }' file.txt       # print fields 1 and 3
awk '{ print NF }' file.txt           # NF = number of fields on this line
awk '{ print NR, $0 }' file.txt       # NR = current line/record number, $0 = whole line
awk -F',' '{ print $2 }' data.csv     # -F sets field separator (comma, for CSV)
awk -F':' '{ print $1 }' /etc/passwd  # extract usernames from /etc/passwd
```

Filtering with patterns

```
awk '$3 > 100' data.txt               # print lines where field 3 numerically exceeds 100
awk '/ERROR/ { print $0 }' app.log    # combine regex pattern with an action
awk '$1 == "sensor_01"' data.csv     # exact field match
awk -F',' '$2 > 25.0 { print $1, $2 }' temps.csv # print sensor ID + temp where temp > 25.0
```

Built-in variables you'll actually use

\$0	entire current line
\$1..\$N	individual fields
NF	number of fields in current line
NR	current line number (across whole input)
FS	input field separator (default: whitespace)
OFS	output field separator (default: single space)

Aggregation (this is where awk earns its keep)

```
awk '{ sum += $3 } END { print sum }' data.txt           # sum field 3 across all lines
awk '{ sum += $2; count++ } END { print sum/count }' data.csv # average of field 2
awk -F',' '{ print $1 }' data.csv | sort | uniq -c | sort -rn # frequency count of a column (classic combi
```

Reformatting output

```
awk -F',' '{ print $1 ":" $2 }' data.csv                # reassemble fields with a custom separator
awk 'BEGIN{OFS="\t"} { print $1, $2 }' data.csv         # set output separator to tab
awk '{ $1=""; print }' file.txt                         # blank out field 1, print the rest (crude column re
```

Practical: parsing a serial/sensor CSV log

```
# file: sensor_01,2026-07-20T10:00:00,23.5,45.2
awk -F',' '{ printf "%-12s %s  temp=%.1fC hum=%.1f%%\n", $1, $2, $3, $4 }' sensor_data.csv
awk -F',' '$3 > 30.0 { print "ALERT:", $1, "overtemp", $3 }' sensor_data.csv
```

Key takeaways

- **awk -F',' '{ print \$N }'** is your instant CSV column extractor — faster than opening a spreadsheet.
 - **{ action } END { final action }** pattern is how you aggregate (sum, average, count) over a whole file.
 - **sort | uniq -c | sort -rn** piped after an awk field extraction is the standard "frequency table" idiom.
-

DAY 9 — Pipes, Redirection & xargs

Concept

Unix philosophy: small tools, connected by pipes, each doing one thing. Every process has three standard streams — stdin (0), stdout (1), stderr (2) — and redirection lets you rewire them.

Redirection

```
command > file.txt          # redirect stdout to file, OVERWRITE
command >> file.txt         # redirect stdout to file, APPEND
command < file.txt          # redirect file content as stdin to command
command 2> errors.txt       # redirect stderr only
command > out.txt 2>&1      # redirect stdout to file, then merge stderr into stdout (ORDER MATTERS)
command 2>&1 > out.txt      # WRONG common mistake: stderr goes to old stdout (terminal), then stdout
command &> out.txt          # bash shorthand: redirect both stdout AND stderr to file
command > /dev/null 2>&1    # discard everything — silence a command completely
command 2> /dev/null       # discard only errors, keep normal output visible
```

Pipes

```
ps aux | grep mosquito    # chain: output of ps becomes input of grep
cat access.log | grep "404" | wc -l # count 404s in a log
dmesg | tail -20          # last 20 kernel messages
ls -la | sort -k5 -n      # sort directory listing by size (field 5, numeric)
```

tee — split a stream to a file AND the screen

```
some_command | tee output.log      # see output live, also save it
some_command | tee -a output.log   # append instead of overwrite
long_build.sh | tee build.log | grep -i error # save everything, but only show errors live
```

xargs — turn stdin lines into command arguments

Many commands don't read stdin as a list of arguments (e.g., `rm` wants args on the command line, not piped text).

`xargs` bridges that gap.

```
find . -name "*.tmp" | xargs rm      # delete every .tmp file found
find . -name "*.log" | xargs -I{} mv {} archive/ # -I{} = placeholder, use for each item individually
echo "file1 file2 file3" | xargs -n 1 echo # -n 1 = one argument per invocation
find . -name "*.py" | xargs grep -l "import serial" # search matched files
find . -name "*.tmp" -print0 | xargs -0 rm # -print0/-0: null-delimited, SAFE for filenames with spaces
```

Always prefer `-print0 / xargs -0` when filenames might contain spaces or newlines — plain pipes split on whitespace and will silently mishandle them.

Command substitution & chaining

```
echo "Today is $(date)"          # command substitution: inline a command's output
files=$(ls *.txt)                # capture output into a variable
cmd1 && cmd2                      # run cmd2 only if cmd1 succeeded (exit code 0)
cmd1 || cmd2                    # run cmd2 only if cmd1 failed
cmd1 ; cmd2                     # run both regardless of success/failure
```

Key takeaways

- `stdout=1, stderr=2` — `2>&1` must come AFTER the stdout redirect to merge correctly.
- `tee` when you need to both watch and save output.

- `xargs -0` with `find -print0` is the safe, correct pattern for bulk file operations — don't parse plain `ls/find` output for filenames with special characters.
-

DAY 10 — Process Management

Concept

Every running program is a process with a PID, a parent, a state, and signals it can receive. Managing processes means finding them, understanding their state, and communicating with them via signals — not just killing them.

Viewing processes

```
ps aux                # all processes, all users, full format (BSD style)
ps -ef               # all processes, full format (System V style) — same info, different columns
ps aux | grep mosquito # find a specific process
ps -o pid,ppid,cmd -p 1234 # custom columns for a specific PID

top                  # live, refreshing process viewer, sorted by CPU by default
htop                 # nicer, colorized, interactive top (install: apt install htop)
# inside top: 'q' quit, 'k' kill (prompts for PID), 'M' sort by memory, 'P' sort by CPU, 'r' renice

pstree               # show process tree (parent/child relationships)
pgrep mosquito       # print PID(s) matching a name, no need to grep ps output
pkill mosquito       # kill by name (careful — matches by process name substring)
```

Process states (seen in `ps` **STAT** column / `top`)

```
R  running or runnable
S  sleeping (waiting on I/O or event, interruptible)
D  uninterruptible sleep (usually waiting on disk I/O — can't even be killed with -9)
Z  zombie (finished, but parent hasn't reaped its exit status yet)
T  stopped (e.g., via Ctrl+Z or SIGSTOP)
```

Signals — the actual mechanism behind "killing" a process

```
kill 1234             # sends SIGTERM (15) — polite request to terminate, process can catch/ignore
kill -9 1234          # sends SIGKILL (9) — cannot be caught or ignored, kernel force-terminates
kill -HUP 1234        # sends SIGHUP (1) — traditionally "reload config", many daemons use it
kill -STOP 1234       # pause a process (SIGSTOP)
kill -CONT 1234       # resume a paused process (SIGCONT)
kill -l              # list all signal names/numbers
killall mosquito      # kill all processes matching that exact name
```

Practice: always try plain `kill` (SIGTERM) first, so the process can clean up (close files, flush buffers, release locks). Reach for `kill -9` only when a process is genuinely unresponsive — it skips all cleanup.

Job control (foreground/background within a shell session)

```
long_running_script.sh & # start in background, shell prompt returns immediately
jobs                     # list background jobs in this shell session
fg                       # bring most recent background job to foreground
fg %2                    # bring job number 2 to foreground
bg %1                    # resume a stopped job in the background
Ctrl+Z                  # suspend the current foreground process (SIGTSTP)
Ctrl+C                  # interrupt/terminate the current foreground process (SIGINT)
```

Surviving logout — detaching a process from the terminal

```
nohup ./monitor.sh &    # ignore SIGHUP; process keeps running after you log out; output -> nohup.out
nohup ./monitor.sh > monitor.log 2>&1 & # same, but redirect output explicitly (recommended)
disown                  # remove a job from the shell's job table so it won't get SIGHUP on exit
```


In practice, for anything long-running and important, you'll actually want `systemd` (Day 12) or `tmux/screen`, not raw `nohup` — but understanding `nohup` explains *why* those tools exist.

Resource snapshot

```
free -h          # memory usage, human-readable
uptime          # load average (1/5/15 min) + uptime
nproc           # number of CPU cores available
```

Key takeaways

- `kill` sends signals, not "kill commands" — `SIGTERM` (15, default) is a request; `SIGKILL` (9) is a mandate with no cleanup.
 - Zombie (Z) processes indicate a parent that isn't reaping children — usually harmless individually, but worth investigating if they accumulate.
 - `nohup` `...` `&` survives logout; a real service belongs in `systemd`.
-

DAY 11 — Package Management (Debian/APT)

Concept

Debian/Ubuntu (and Raspberry Pi OS) use **dpkg** as the low-level package tool and **apt/apt-get** as the high-level dependency-resolving front end. You'll use **apt** almost exclusively day-to-day.

apt — the daily driver

```
sudo apt update           # refresh the package index (does NOT install/upgrade anything)
sudo apt upgrade          # upgrade all installed packages to latest available versions
sudo apt full-upgrade     # like upgrade, but will also remove packages if needed to resolve
sudo apt install mosquitto mosquitto-clients # install one or more packages
sudo apt remove mosquitto # remove package, keep config files
sudo apt purge mosquitto  # remove package AND its config files
sudo apt autoremove       # remove packages that were auto-installed as dependencies and
apt search mqtt           # search package names/descriptions
apt show mosquitto        # detailed info about a package (version, deps, descriptions)
apt list --installed      # list all installed packages
apt list --installed | grep python # find installed packages matching a name
```

dpkg — lower level, direct package file operations

```
sudo dpkg -i somepackage.deb # install a local .deb file directly
dpkg -l                      # list all installed packages (dpkg's own format)
dpkg -l | grep mosquitto     # check if a specific package is installed
dpkg -L mosquitto            # list all FILES installed by a package (where did it put things?)
dpkg -S /usr/bin/mosquitto   # reverse lookup: which package owns this file?
```

Practical workflow: setting up an MQTT broker on a fresh Raspberry Pi

```
sudo apt update
sudo apt install -y mosquitto mosquitto-clients
sudo systemctl enable --now mosquitto # (preview of Day 12) enable + start immediately
mosquitto_pub -h localhost -t test/topic -m "hello"
mosquitto_sub -h localhost -t test/topic
```

Holding a package at its current version (avoid unwanted upgrades)

```
sudo apt-mark hold mosquitto # prevent this package from being upgraded
sudo apt-mark unhold mosquitto # allow upgrades again
apt-mark showhold            # list all held packages
```

Cleaning up

```
sudo apt clean # remove all cached .deb files from /var/cache/apt/archives
sudo apt autoclean # remove only cached files for packages no longer installable (safer, partial)
```

Key takeaways

- **apt update** = refresh index; **apt upgrade** = actually install newer versions. They are two different actions, always run together in that order.
 - **dpkg -L <pkg>** / **dpkg -S <path>** is your two-way lookup between packages and files — invaluable when debugging "why is this binary here" or "what did installing X actually put on disk."
 - **apt-mark hold** protects a known-good version on a production/embedded device from being clobbered by an unattended upgrade.
-

DAY 12 — systemd: Services, Units & journalctl

Concept

systemd is PID 1 on virtually all modern Linux distros — it manages service startup, dependencies, restarts, and logging. For anything you want running reliably (like your `mqtt_monitor`), a systemd unit is the correct home, not a background shell script.

Controlling services

```
sudo systemctl start mosquitto      # start now
sudo systemctl stop mosquitto       # stop now
sudo systemctl restart mosquitto    # stop then start
sudo systemctl reload mosquitto     # ask service to reload config without restarting (if supported)
sudo systemctl status mosquitto     # current state, recent log lines, main PID
sudo systemctl enable mosquitto     # start automatically on boot (creates symlinks, doesn't start now)
sudo systemctl disable mosquitto    # don't start on boot
sudo systemctl enable --now mosquitto # enable AND start in one command
systemctl is-active mosquitto       # just print active/inactive
systemctl is-enabled mosquitto      # just print enabled/disabled
systemctl list-units --type=service --state=running # what's currently running
systemctl list-units --type=service --state=failed  # what's broken
```

Writing your own unit file

```
sudo nano /etc/systemd/system/mqtt_monitor.service

[Unit]
Description=MQTT Device Monitor Service
After=network.target mosquitto.service
Wants=mosquitto.service

[Service]
Type=simple
User=monitoruser
Group=dialout
WorkingDirectory=/opt/mqtt_monitor
ExecStart=/opt/mqtt_monitor/venv/bin/python3 /opt/mqtt_monitor/main.py
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal
Environment=PYTHONUNBUFFERED=1

[Install]
WantedBy=multi-user.target
```

Then:

```
sudo systemctl daemon-reload # REQUIRED after creating/editing any unit file
sudo systemctl enable --now mqtt_monitor
sudo systemctl status mqtt_monitor
```

Key directives explained

- `After=` / `Wants=` — ordering and soft dependency (won't fail to start if the dependency is down, but waits for it)
- `Requires=` — hard dependency (this unit fails if the dependency fails)
- `Type=simple` — process doesn't fork; the `ExecStart` process IS the service (most common for scripts/daemons written this way)

- **Type=forking** — traditional daemon that forks and exits the parent
- **Restart=on-failure** — auto-restart if it exits non-zero; **Restart=always** also restarts on clean exit
- **WantedBy=multi-user.target** — this is what **enable** actually links into, controlling boot-time startup

journalctl — the systemd log viewer

```
journalctl -u mqtt_monitor          # all logs for this unit
journalctl -u mqtt_monitor -f       # follow live (like tail -f, but for the journal)
journalctl -u mqtt_monitor --since today    # today's logs only
journalctl -u mqtt_monitor --since "1 hour ago"
journalctl -u mqtt_monitor -n 50      # last 50 lines
journalctl -p err                    # only priority "err" and above, across all units
journalctl -b                        # logs since last boot
journalctl -b -1                     # logs from the PREVIOUS boot
journalctl --disk-usage               # how much space the journal is consuming
sudo journalctl --vacuum-time=7d      # delete journal entries older than 7 days
sudo journalctl --vacuum-size=200M    # cap total journal size
```

Timers (systemd's cron alternative — previewed here, detailed Day 22)

```
systemctl list-timers                # show all active systemd timers and next run time
```

Key takeaways

- **daemon-reload** after every unit file edit, or systemd won't see the change — this is the #1 "why isn't my fix working" gotcha.
 - **Restart=on-failure + RestartSec=** gives you free auto-recovery for a service that occasionally crashes.
 - **journalctl -u <name> -f** replaces "tail -f a log file" for anything running under systemd.
-

DAY 13 — Networking Basics

Concept

Command-line networking diagnostics answer three questions in order: is the interface up, can I reach the host, and can I reach the service/port. Work through them in that order when debugging connectivity.

Interface & address inspection

```
ip addr show          # (aka `ip a`) all interfaces, IP addresses, state (modern replacement for ifconfig)
ip a show eth0        # just one interface
ip link show          # interfaces and link state (up/down), without IP details
ip route show         # (aka `ip r`) routing table – what gateway is used for what destination
hostname              # this machine's hostname
hostname -I           # all IP addresses assigned to this machine, quick and dirty
```

Reachability

```
ping 8.8.8.8          # ICMP echo – is the host reachable at the network layer?
ping -c 4 192.168.1.10 # send exactly 4 pings then stop (don't let it run forever)
ping -c 4 broker.local # test hostname resolution + reachability together
traceroute 8.8.8.8     # show the route (hop by hop) packets take to a destination
mtr 8.8.8.8           # combines ping + traceroute, live/continuous (install: apt install mtr)
```

DNS

```
cat /etc/resolv.conf  # currently configured DNS servers
nslookup broker.local # basic DNS lookup
dig broker.local       # detailed DNS lookup (more info than nslookup)
dig +short broker.local # just the resolved IP, no extra output
getent hosts broker.local # resolve using the full system resolution chain (incl. /etc/hosts)
```

Ports & connections

```
ss -tuln              # (modern replacement for netstat) TCP/UDP listening sockets, numeric, no names
ss -tulnp             # same, plus the PID/process name owning each socket (needs sudo for other users)
ss -tan               # all TCP connections (not just listening), numeric
sudo netstat -tulnp   # older tool, same idea as ss -tulnp, still common in the wild
```

Practical: to check whether the MQTT broker is actually listening on port 1883:

```
ss -tulnp | grep 1883
```

Fetching data over HTTP

```
curl https://api.example.com/status # GET request, print response body to stdout
curl -s https://api.example.com/status # silent (no progress meter)
curl -I https://api.example.com      # HEAD request – just headers, fast status check
curl -o output.json https://api.example.com/data # save response to a file
curl -X POST -d '{"key":"value"}' -H "Content-Type: application/json" https://api.example.com/data
curl -v https://api.example.com      # verbose: show full request/response, headers, status
wget https://example.com/firmware.bin # download a file (better default for straight downloads)
wget -c https://example.com/large_file.iso # resume a partial download
```

MQTT-specific connectivity check (ties directly to your stack)

```
mosquitto_sub -h broker.local -t 'devices/#' -v # subscribe to everything under devices/, print topic and payload
mosquitto_pub -h broker.local -t test -m "ping" # publish a test message
nc -zv broker.local 1883 # is TCP port 1883 open/reachable? (see Day 26 for MQTT details)
```

Key takeaways

- Diagnostic order: interface up (`ip a`) → host reachable (`ping`) → port open (`ss/nc`) → application responds (`curl/mosquitto_sub`).
 - `ss` has replaced `netstat` on modern systems — learn `ss -tulnp` as your default.
 - `curl -v` and `dig` give you the verbose truth when something "should be working" but isn't.
-

DAY 14 — SSH & Remote Access

Concept

SSH is how you administer remote/embedded Linux boxes in practice — encrypted remote shell, file transfer, port forwarding, and key-based auth that eliminates passwords entirely.

Basic connection

```
ssh george@192.168.1.50      # connect as user george to that host
ssh george@raspberrypi.local # mDNS hostname (works out of the box on Raspberry Pi OS)
ssh -p 2222 george@host      # connect on a non-default port
exit                          # end the SSH session (or Ctrl+D)
```

Key-based authentication (do this instead of passwords)

```
ssh-keygen -t ed25519 -C "george@workstation" # generate a keypair (ed25519 = modern, fast, secure d
# creates ~/.ssh/id_ed25519 (PRIVATE – never share) and ~/.ssh/id_ed25519.pub (public – safe to distribute)

ssh-copy-id george@192.168.1.50                # copy your public key to the remote host's authorize
# or manually:
cat ~/.ssh/id_ed25519.pub | ssh george@host 'cat >> ~/.ssh/authorized_keys'

chmod 700 ~/.ssh                                # required permissions or SSH will refuse to use the directory
chmod 600 ~/.ssh/authorized_keys                # required permissions on the key file itself
```

SSH config file — stop typing full connection strings

`~/.ssh/config`:

```
Host rpi-monitor
  HostName 192.168.1.50
  User george
  Port 22
  IdentityFile ~/.ssh/id_ed25519

Host beaglebone
  HostName 192.168.7.2
  User debian
  IdentityFile ~/.ssh/id_ed25519
```

Then simply: `ssh rpi-monitor` — no need to remember IPs, usernames, or ports.

File transfer

```
scp file.txt george@host:/home/george/      # copy local file to remote
scp george@host:/var/log/app.log ./          # copy remote file to local
scp -r localdir/ george@host:/remote/path/   # recursive directory copy

rsync -avz localdir/ george@host:/remote/path/ # preferred over scp for anything repeated: only t
rsync -avz --delete localdir/ george@host:/remote/path/ # also delete files on remote that no longer ex
rsync -avz george@host:/remote/logs/ ./local_logs/ # pull logs down from a device for analysis
```

`rsync` flags: `-a` = archive (preserves perms/times/links, recursive), `-v` = verbose, `-z` = compress during transfer.

Running remote commands without a full interactive session

```
ssh george@host 'systemctl status mqtt_monitor' # run one command remotely, see output locally, then c
ssh george@host 'df -h' > remote_disk_usage.txt # capture remote output into a local file
```

ssh-agent — avoid retyping your key passphrase

```
eval "$(ssh-agent -s)"          # start the agent for this shell session
ssh-add ~/.ssh/id_ed25519      # load your key into the agent (enter passphrase once)
ssh-add -l                     # list keys currently loaded in the agent
```

Port forwarding (useful for reaching a service on an embedded box behind NAT/firewall)

```
ssh -L 8080:localhost:80 george@host    # local port forward: localhost:8080 -> host's localhost:80
ssh -R 9000:localhost:3000 george@host  # remote port forward: host's 9000 -> your localhost:3000
```

Key takeaways

- Key-based auth + `~/.ssh/config` aliases eliminate almost all SSH friction — set this up once per host.
 - `rsync -avz` over `scp` whenever you're syncing more than a one-off file, especially repeatedly.
 - `.ssh` directory must be `700` and `authorized_keys`/private keys must be `600`, or SSH silently refuses them.
-

DAY 15 — Device Files & Serial Communication (Embedded Focus)

Concept

On Linux, hardware is represented as files under `/dev`. Serial devices, USB devices, and GPIO all surface here or under `/sys`. This is where your embedded/IoT background connects directly to the shell.

Discovering devices

```
ls /dev/tty*           # all terminal/serial devices
ls /dev/ttyUSB*        # USB-to-serial adapters (FTDI, CP210x, CH340, etc.)
ls /dev/ttyACM*        # USB CDC-ACM devices (Arduino, many microcontroller dev boards)
ls /dev/ttyS*          # native/onboard serial ports (e.g. Raspberry Pi's UART, if enabled)

lsusb                  # list all USB devices currently connected
lsusb -v               # verbose: full descriptor info (vendor/product IDs, interfaces)
lsusb -t               # tree view showing USB hub/hierarchy

dmesg | tail -20       # kernel ring buffer – plug in a USB-serial device and watch this to see if
dmesg -w               # follow kernel messages live (like tail -f but for dmesg) – plug in a device
dmesg | grep -i tty    # filter for tty-related kernel messages
dmesg | grep -i usb    # filter for USB enumeration messages
```

Practical device discovery workflow (the sequence you'll actually run)

```
ls /dev/ttyUSB* /dev/ttyACM* 2>/dev/null # what's there before plugging in
# plug in the device
dmesg | tail -10              # confirm it enumerated and see which /dev/tty* it became
ls -la /dev/ttyUSB0           # confirm the device node exists and check its group owners
```

udev — persistent device naming (critical for production embedded setups)

The problem: `/dev/ttyUSB0` vs `/dev/ttyUSB1` can swap between reboots if you have multiple USB-serial adapters.
Fix: create a udev rule that assigns a fixed symlink based on the device's unique attributes.

```
udevadm info -a -n /dev/ttyUSB0 | less # dump all attributes of this device (find serial number, vendor, etc.)
```

Create `/etc/udev/rules.d/99-usb-serial.rules`:

```
SUBSYSTEM=="tty", ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6001", ATTRS{serial}=="AB0JFH9", SYMLINK+="sensor_gateway"

sudo udevadm control --reload-rules # reload rules without rebooting
sudo udevadm trigger                # re-apply rules to currently connected devices
ls -la /dev/sensor_gateway           # now points reliably to the right ttyUSB device regardless of reboot
```

This means your `mqtt_monitor` config can reference `/dev/sensor_gateway` instead of a `/dev/ttyUSB0` that might silently become `/dev/ttyUSB1` after a reboot.

Raw serial interaction from the command line

```
stty -F /dev/ttyUSB0 115200 cs8 -cstopb -parenb # set baud rate 115200, 8N1 framing
stty -F /dev/ttyUSB0 -a                          # show ALL current settings for that serial line

cat /dev/ttyUSB0 # crude: dump raw incoming serial data to terminal (Ctrl+C to stop)
echo "AT" > /dev/ttyUSB0 # crude: send raw data out the serial line

screen /dev/ttyUSB0 115200 # interactive serial terminal (Ctrl+A then K to kill session)
minicom -D /dev/ttyUSB0 -b 115200 # more full-featured serial terminal (apt install minicom)
```

Key takeaways

- `dmesg -w` while plugging in hardware is the fastest way to identify which `/dev/tty*` node a device became.
 - Use a udev rule with vendor/product/serial matching to get a stable device name — don't hardcode `/dev/ttyUSB0` in production configs.
 - `stty -F <device> -a` before debugging "garbage on the serial line" — 9 times out of 10 it's a baud rate or framing mismatch.
-

DAY 16 — Archiving, Compression & Backups

Concept

tar bundles multiple files into one archive (originally for tape drives, hence the name); compression is a separate, composable step. **rsync** is the practical tool for real backups because it's incremental.

tar

```
tar -cvf archive.tar dir/      # create (c), verbose (v), file (f) – plain, uncompressed tarball
tar -xvf archive.tar          # extract
tar -tvf archive.tar          # list contents WITHOUT extracting (always do this before extracting)

tar -czvf archive.tar.gz dir/  # create + gzip compress in one step (z = gzip)
tar -xzvf archive.tar.gz      # extract a .tar.gz
tar -cjvf archive.tar.bz2 dir/ # create + bzip2 compress (j) – better ratio, slower
tar -cJvf archive.tar.xz dir/  # create + xz compress (J) – best ratio, slowest

tar -czvf backup.tar.gz --exclude='*.log' dir/ # exclude a pattern from the archive
tar -xzvf archive.tar.gz -C /target/directory/ # extract into a specific target directory
```

gzip/zip family

```
gzip file.txt                # compress in place -> file.txt.gz (original is removed)
gunzip file.txt.gz           # decompress in place
zcat file.txt.gz             # view compressed file content without fully decompressing to disk

zip archive.zip file1 file2   # create a zip (cross-platform friendly, unlike tar)
zip -r archive.zip directory/ # recursive zip of a directory
unzip archive.zip             # extract
unzip -l archive.zip          # list contents without extracting
```

rsync for real backups (incremental, efficient, resumable)

```
rsync -av /home/george/project/ /mnt/backup/project/ # local backup, archive mode, verbose
rsync -avz george@rpi:/opt/mqtt_monitor/ ./mqtt_monitor_backup/ # pull a remote directory to local backup
rsync -av --delete src/ dst/ # mirror mode: dst becomes an exact copy of src
rsync -av --dry-run src/ dst/ # PREVIEW what would change, without actually changing anything
rsync -avP src/ dst/         # -P = show progress + allow resume
```

Practical scheduled backup pattern (paired with cron on Day 22):

```
rsync -av --delete /opt/mqtt_monitor/data/ /mnt/backup/mqtt_monitor_$(date +%F)/
```

Key takeaways

- **tar -tvf** before **tar -xvf** — always preview an unfamiliar archive's contents before extracting it somewhere.
 - Choose compression by tradeoff: gzip (fast, decent), xz (slow, best ratio) — for embedded devices with limited CPU, gzip is usually the pragmatic default.
 - **rsync -av --dry-run** before any **--delete** run — this single habit prevents the worst rsync accidents.
-

DAY 17 — Disks, Filesystems & Storage Management

Concept

Understanding block devices, partitions, filesystems, and mount points is essential on embedded targets where you're often working with SD cards, USB storage, or managing limited flash space.

Inspecting storage

```
df -h                # disk free space, human-readable, per mounted filesystem
df -i                # inode usage (you can run out of inodes even with free space – common on sy
du -sh directory/    # total size of a directory (summarized, human-readable)
du -sh */             # size of each subdirectory in current dir – great for hunting what's ea
du -h --max-depth=1 /var/ | sort -rh  # top-level breakdown of /var, sorted largest first

lsblk                # list block devices as a tree: disks -> partitions -> mount points
lsblk -f             # same, plus filesystem type and UUID
fdisk -l             # list partition tables (needs sudo typically)
blkid                # list filesystem UUIDs and types for all block devices
```

Mounting

```
sudo mount /dev/sdb1 /mnt/usb          # mount a partition at a mount point (mount point must already exist)
sudo umount /mnt/usb                   # unmount – do this before physically removing the device
sudo umount /dev/sdb1                  # can unmount by device path too
mount | column -t                       # show all current mounts, nicely aligned
findmnt /mnt/usb                        # detailed info about what's mounted where
```

/etc/fstab — mounts that persist across reboots

```
UUID=1234-5678 /mnt/data ext4 defaults 0 2

cat /etc/fstab                # view configured persistent mounts
sudo mount -a                  # mount everything listed in fstab (also validates it without rebo
blkid /dev/sda1                # get the UUID to put into fstab (prefer UUID over /dev/sdX – dev
```

Practical caution: a broken `/etc/fstab` entry can prevent a system from booting to a usable state — always test with `mount -a` before rebooting, and keep a backup of the file before editing.

Partitioning & filesystem creation (SD card prep — directly relevant for RPi/BeagleBone work)

```
sudo fdisk /dev/sdb          # interactive partition editor (n=new, d=delete, p=print, w=write) – DEVI
sudo parted /dev/sdb print    # alternative partition tool, often used for GPT
sudo mkfs.ext4 /dev/sdb1      # format a partition as ext4
sudo mkfs.vfat -F 32 /dev/sdb1 # format as FAT32 (needed for the boot partition on RPi images)
```

Disk usage triage workflow (when a device is running out of space)

```
df -h                # which filesystem is full?
du -h --max-depth=1 / 2>/dev/null | sort -rh  # which top-level directory is the culprit?
du -h --max-depth=1 /var/log 2>/dev/null | sort -rh # drill down (logs are the usual suspect)
journalctl --disk-usage    # check if the systemd journal itself is the problem
sudo journalctl --vacuum-size=100M    # reclaim space from journal if so
```

Key takeaways

- `df` shows filesystem-level free space; `du` shows how a directory tree's actual usage breaks down — you need both to diagnose "disk full."

- Use UUIDs (not `/dev/sdX` names) in `/etc/fstab` — device letters can reorder between boots.
 - On embedded targets, `journalctl --vacuum-size` and log rotation (Day 23) are often the fix for a filling flash storage.
-

DAY 18 — Shell Environment & Configuration

Concept

Your shell's behavior — prompt, aliases, PATH, environment variables — is configured through a predictable set of startup files, read in a specific order depending on how the shell is invoked.

The startup file chain (bash)

/etc/profile	system-wide, read by login shells
~/.bash_profile	per-user, login shells (if it exists, ~/.profile is skipped)
~/.profile	per-user, login shells (fallback if .bash_profile absent)
~/.bashrc	per-user, NON-login interactive shells (most new terminal windows/tabs)
/etc/bash.bashrc	system-wide equivalent of .bashrc

Practical rule of thumb: put PATH/environment exports that should apply everywhere in **.profile**; put aliases, functions, and prompt customization in **.bashrc**. Most terminal emulators open non-login shells, so **.bashrc** is what actually gets read most of the time — that's why so many guides tell you to "just add it to .bashrc."

Environment variables

env	# list all environment variables
printenv PATH	# print one specific variable
echo \$HOME	# \$VAR syntax to reference a variable's value
export MY_VAR="value"	# set AND export (make visible to child processes) in one step
MY_VAR="value"	# sets a shell variable, but does NOT export it – child processes won't
unset MY_VAR	# remove a variable

PATH — how the shell finds executables

echo \$PATH	# colon-separated list of directories searched, in order, for commands
export PATH="\$HOME/bin:\$PATH"	# prepend a personal bin directory (found FIRST, can shadow system commands)
export PATH="\$PATH:/opt/tools"	# append a directory (found LAST)
which python3	# shows which PATH entry actually gets used
type -a python3	# shows ALL matches across PATH, in search order (reveals shadowing)

Aliases and functions

```
alias ll='ls -la' # simple command shorthand
alias gs='git status'
alias mqttlog='journalctl -u mqtt_monitor -f'

# functions handle arguments, which aliases can't do well:
mkcd() {
    mkdir -p "$1" && cd "$1"
}
```

Put these in **~/.bashrc** to persist across sessions. After editing:

source ~/.bashrc	# re-read the file into the CURRENT shell without opening a new terminal
. ~/.bashrc	# equivalent shorthand for `source`

Common practical additions to **.bashrc** for a dev/embedded workflow

export EDITOR=nano	# default editor for tools that invoke \$EDITOR (e.g. c
export HISTSIZE=10000	# keep more shell history in memory
export HISTFILESIZE=20000	# keep more shell history on disk
export HISTCONTROL=ignoredups:erasedups	# don't clutter history with duplicate consecutive
shopt -s histappend	# append to history file instead of overwriting
alias serial='minicom -D /dev/sensor_gateway -b 115200'	# project-specific shortcut

Shell variable scope quirks

```
VAR=hello
bash -c 'echo $VAR'           # prints nothing – VAR wasn't exported, child process (new bash) can't see
export VAR=hello
bash -c 'echo $VAR'           # prints "hello" – now it's in the environment, inherited by children
```

Key takeaways

- `.bashrc` for aliases/functions/prompt, `.profile` for PATH/env exports meant to apply to login shells too — but for a single-user dev machine, put it all in `.bashrc` and don't overthink it.
 - `export` is the difference between a shell variable (local to this shell) and an environment variable (inherited by child processes).
 - `source ~/.bashrc` after every edit — forgetting this is the most common "why isn't my alias working" moment.
-

DAY 19 — Shell Scripting I: Fundamentals

Concept

A shell script is just a sequence of commands you'd type interactively, saved to a file and made executable. The fundamentals — shebang, variables, arguments, exit codes — are what every script builds on.

Anatomy of a script

```
#!/usr/bin/env bash
# ^ shebang: tells the OS which interpreter runs this file.
# `#!/usr/bin/env bash` is more portable than `#!/bin/bash` (finds bash on $PATH, works if bash lives somewhere)

set -euo pipefail
# -e : exit immediately if any command fails (non-zero exit code)
# -u : treat unset variables as an error instead of silently expanding to empty string
# -o pipefail : a pipeline's exit code is the first non-zero one, not just the last command's
# This trio is close to a non-negotiable default for any real script.

echo "Starting monitor check..."
```

Make it executable and run it:

```
chmod +x check_monitor.sh
./check_monitor.sh          # note the ./ – required unless the script's directory is on $PATH
bash check_monitor.sh       # alternative: explicitly invoke the interpreter, ignores the shebang
```

Variables

```
NAME="mqtt_monitor"
echo "$NAME"          # ALWAYS quote variable expansions: "$NAME", not $NAME (prevents word-splitting)
echo "${NAME}_service" # braces disambiguate where the variable name ends
readonly VERSION="1.0" # constant, cannot be reassigned

COUNT=0
COUNT=$((COUNT + 1)) # arithmetic: $(( )) for integer math
COUNT=$((COUNT + 1))
```

Command-line arguments

```
#!/usr/bin/env bash
echo "Script name: $0"
echo "First arg:   $1"
echo "Second arg:  $2"
echo "All args:    @"
echo "Arg count:   $#"
```



```
if [ $# -lt 1 ]; then
    echo "Usage: $0 <device_path>" >&2
    exit 1
fi
```



```
DEVICE="$1"
echo "Using device: $DEVICE"
```



```
./check_device.sh /dev/ttyUSB0
```

Exit codes — the real return value of every command and script

```
echo $?          # exit code of the LAST command run (0 = success, non-zero = some kind of failure)
ls /nonexistent; echo $? # will print a non-zero code, typically 2
```



```
exit 0                # explicitly succeed
exit 1                # explicitly fail (generic)
```

Convention: 0 = success, 1-255 = various failure meanings (your own scripts can define what non-zero codes mean; many tools follow loose conventions like 1=general error, 2=misuse of command).

Reading user input

```
read -p "Enter device path: " DEVICE_PATH
echo "You entered: $DEVICE_PATH"

read -s -p "Enter password: " PASSWORD    # -s = silent, don't echo typed characters
echo
```

Command substitution inside scripts

```
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
LOGFILE="monitor_${TIMESTAMP}.log"
echo "Logging to $LOGFILE"

HOSTNAME=$(hostname)
UPTIME_SECONDS=$(cut -d' ' -f1 /proc/uptime)
```

Key takeaways

- `#!/usr/bin/env bash + set -euo pipefail` at the top of nearly every script you write from here on.
 - Always quote variable expansions: `"$VAR"` — unquoted expansion is the single most common source of shell scripting bugs (word splitting, glob expansion on the value).
 - `$?` is the exit code of the last command — check it (or let `set -e` check it for you) rather than assuming success.
-

DAY 20 — Shell Scripting II: Control Flow

Concept

Conditionals and loops turn a linear script into real logic. Bash's test syntax (`[ ]` / `[[ ]]`) has quirks worth knowing precisely, since subtle mistakes here are a classic source of bugs.

if / elif / else

```
#!/usr/bin/env bash
DEVICE="/dev/ttyUSB0"

if [ -e "$DEVICE" ]; then
    echo "Device exists"
elif [ -e "/dev/ttyACM0" ]; then
    echo "Falling back to ttyACM0"
else
    echo "No serial device found" >&2
    exit 1
fi
```

Test operators you'll actually use

```
# File tests
[ -e "$FILE" ]      # exists
[ -f "$FILE" ]      # exists AND is a regular file
[ -d "$DIR" ]       # exists AND is a directory
[ -r "$FILE" ]      # readable
[ -w "$FILE" ]      # writable
[ -x "$FILE" ]      # executable
[ -s "$FILE" ]      # exists and has size > 0 (non-empty)
[ -L "$FILE" ]      # is a symlink

# String tests
[ -z "$STR" ]       # string is empty
[ -n "$STR" ]       # string is non-empty
[ "$A" = "$B" ]     # string equality (single = , POSIX-compatible)
[ "$A" != "$B" ]    # string inequality

# Numeric comparisons (NOTE: different operators than strings!)
[ "$A" -eq "$B" ]   # equal
[ "$A" -ne "$B" ]   # not equal
[ "$A" -lt "$B" ]   # less than
[ "$A" -le "$B" ]   # less than or equal
[ "$A" -gt "$B" ]   # greater than
[ "$A" -ge "$B" ]   # greater than or equal

# Logical combination
[ "$A" -eq 1 ] && [ "$B" -eq 2 ]    # AND — two separate test invocations chained
[ "$A" -eq 1 ] || [ "$B" -eq 2 ]    # OR
```

[[]] — bash's extended test (preferred over [] in bash-only scripts)

```
if [[ "$STR" == *"error"* ]]; then    # pattern matching works directly, no need for external tools
    echo "found error"
fi

if [[ -f "$FILE" && -r "$FILE" ]]; then    # && / || work directly inside [[ ]], unlike [ ]
    echo "readable file"
fi

if [[ "$STR" =~ ^sensor_[0-9]+$ ]]; then    # =~ enables regex matching directly
```

```

        echo "matches sensor naming pattern"
    fi

```

`[[ ]]` is a bash keyword (safer parsing, no word-splitting surprises on unquoted variables, supports `&&/|/!=~` natively). `[ ]` is the POSIX `test` command (portable to `sh`, but stricter/quirkier). For bash scripts specifically (shebang `#!/usr/bin/env bash`), prefer `[[ ]]`.

case statements — cleaner than a long if/elif chain

```

read -p "Action (start/stop/status): " ACTION
case "$ACTION" in
    start)
        systemctl start mqtt_monitor
        ;;
    stop)
        systemctl stop mqtt_monitor
        ;;
    status)
        systemctl status mqtt_monitor
        ;;
    *)
        echo "Unknown action: $ACTION" >&2
        exit 1
        ;;
esac

```

Loops

```

# for loop over a list
for device in /dev/ttyUSB0 /dev/ttyUSB1 /dev/ttyACM0; do
    if [ -e "$device" ]; then
        echo "Found: $device"
    fi
done

# for loop over glob results
for logfile in /var/log/mqtt_monitor/*.log; do
    echo "Processing $logfile"
done

# C-style for loop
for ((i=0; i<10; i++)); do
    echo "Iteration $i"
done

# while loop
COUNT=0
while [ "$COUNT" -lt 5 ]; do
    echo "Count: $COUNT"
    COUNT=$((COUNT + 1))
done

# until loop (opposite of while – runs UNTIL condition becomes true)
until systemctl is-active --quiet mqtt_monitor; do
    echo "Waiting for service to become active..."
    sleep 1
done

# reading a file line by line (the CORRECT, safe pattern)
while IFS= read -r line; do
    echo "Line: $line"
done < input.txt

```

The `while IFS= read -r line; do ... done < file` pattern is the standard, correct way to process a file line-by-line — `IFS=` prevents leading/trailing whitespace trimming, `-r` prevents backslash escapes from being interpreted.

Loop control

<code>break</code>	<code># exit the loop immediately</code>
<code>continue</code>	<code># skip to the next iteration</code>

Key takeaways

- `[[ ]]` over `[ ]` in bash-only scripts — safer, more capable (`&&`, `||`, `=~`, pattern matching all work natively inside it).
 - String comparison uses `=/!=`; numeric comparison uses `-eq/-ne/-lt/etc.` — mixing these up is a very common bug.
 - `while IFS= read -r line; do ... done < file` is the correct file-reading loop; avoid `for line in $(cat file)` (breaks on whitespace, doesn't preserve lines correctly).
-

DAY 21 — Shell Scripting III: Functions, Arrays & Advanced Patterns

Concept

Larger scripts need reusable functions, structured data (arrays), robust string handling, and graceful cleanup — this is where a script starts looking like real software rather than a one-off command sequence.

Functions

```
#!/usr/bin/env bash

check_device() {
    local device="$1"          # 'local' scopes the variable to this function – always use it for functions
    if [ -e "$device" ]; then
        echo "OK: $device found"
        return 0               # explicit success exit code
    else
        echo "MISSING: $device" >&2
        return 1               # explicit failure exit code
    fi
}

if check_device "/dev/ttyUSB0"; then
    echo "Proceeding with monitor startup"
else
    echo "Aborting" >&2
    exit 1
fi

# capturing a function's stdout as a value (functions "return" text via echo, not via 'return' for data)
get_timestamp() {
    date +%Y%m%d_%H%M%S
}
TS=$(get_timestamp)
echo "Timestamp: $TS"
```

Arrays

```
DEVICES=("/dev/ttyUSB0" "/dev/ttyUSB1" "/dev/ttyACM0")

echo "${DEVICES[0]}"          # first element
echo "${DEVICES[@]}"          # all elements
echo "${#DEVICES[@]}"         # array length

for d in "${DEVICES[@]}"; do   # always quote "${ARRAY[@]}" when iterating – preserves elements with spaces
    echo "Checking $d"
done

DEVICES+=("/dev/ttyUSB2")      # append an element

# associative arrays (bash 4+, key-value)
declare -A CONFIG
CONFIG["broker"]="localhost"
CONFIG["port"]="1883"
CONFIG["topic"]="devices/#"

for key in "${!CONFIG[@]}"; do # ! prefix iterates KEYS of an associative array
    echo "$key = ${CONFIG[$key]}"
done
```

String manipulation

```
STR="mqtt_monitor.log"
echo "${STR%.log}"           # strip shortest matching suffix -> "mqtt_monitor"
echo "${STR#mqtt_}"         # strip shortest matching prefix -> "monitor.log"
echo "${STR/monitor/MONITOR}" # replace first match -> "mqtt_MONITOR.log"
echo "${STR//o/O}"          # replace ALL matches -> "mqtt_mOnitOr.lOg"
echo "${#STR}"              # string length
echo "${STR:0:4}"           # substring: start at 0, length 4 -> "mqtt"
echo "${STR^^}"             # uppercase entire string (bash 4+)
echo "${STR,,}"             # lowercase entire string

VAR=""
echo "${VAR:-default}"      # use "default" if VAR is unset OR empty (doesn't modify VAR)
echo "${VAR:=default}"      # same, but ALSO assigns "default" to VAR if it was unset/empty
```

Here-documents — multi-line text blocks

```
cat << EOF > /etc/systemd/system/mqtt_monitor.service
[Unit]
Description=MQTT Device Monitor
After=network.target

[Service]
ExecStart=/opt/mqtt_monitor/main.py
Restart=on-failure

[Install]
WantedBy=multi-user.target
EOF
```

This generates config/unit files programmatically from a script — variables inside the heredoc ARE expanded unless you quote the delimiter (<< 'EOF' disables expansion, useful when the content itself contains \$ literally).

trap — cleanup on exit or signal

```
#!/usr/bin/env bash
LOCKFILE="/tmp/monitor.lock"

cleanup() {
    echo "Cleaning up..."
    rm -f "$LOCKFILE"
}
trap cleanup EXIT          # runs on ANY script exit – normal completion, error, or signal
trap cleanup INT TERM      # explicitly also on Ctrl+C (INT) and kill (TERM), if you want different ha

touch "$LOCKFILE"
echo "Working..."
sleep 10
# cleanup() runs automatically when the script ends, however it ends
```

Putting it together: a realistic device-check script

```
#!/usr/bin/env bash
set -euo pipefail

DEVICES=("/dev/ttyUSB0" "/dev/ttyUSB1" "/dev/ttyACM0")
LOGFILE="/var/log/mqtt_monitor/device_check_$(date +%Y%m%d).log"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*" | tee -a "$LOGFILE"
}
```

```

find_active_device() {
    for device in "${DEVICES[@]}; do
        if [ -e "$device" ]; then
            echo "$device"
            return 0
        fi
    done
    return 1
}

if ACTIVE_DEVICE=$(find_active_device); then
    log "Active device found: $ACTIVE_DEVICE"
else
    log "ERROR: no serial device found among: ${DEVICES[*]}"
    exit 1
fi

```

Key takeaways

- `local` inside every function variable — without it, function variables leak into (and can clobber) the caller's scope.
 - `"${ARRAY[@]}"` (quoted, with `[@]`) is the correct way to expand an array preserving element boundaries.
 - `trap cleanup EXIT` guarantees cleanup code runs regardless of how the script terminates — the correct pattern for lockfiles, temp files, and releasing resources.
-

DAY 22 — Scheduled Tasks: cron & systemd Timers

Concept

Two mechanisms exist for running things on a schedule: **cron** (traditional, simple, ubiquitous) and **systemd timers** (more powerful, integrated with logging/dependencies). Know both — cron for quick/simple/portable jobs, timers for anything that benefits from systemd's dependency and logging model.

cron

```
crontab -l          # list current user's cron jobs
crontab -e          # edit current user's cron jobs (opens $EDITOR)
sudo crontab -e -u george # edit another user's crontab (as root)
sudo crontab -l -u george # list another user's crontab
```

Crontab syntax

```
# minute hour day-of-month month day-of-week  command
* * * * *                                     command_to_run

0 2 * * * /opt/mqtt_monitor/scripts/backup.sh      # every day at 2:00 AM
*/15 * * * * /opt/mqtt_monitor/scripts/health_check.sh # every 15 minutes
0 0 * * 0 /opt/mqtt_monitor/scripts/weekly_report.sh # every Sunday at midnight
0 9-17 * * 1-5 /opt/mqtt_monitor/scripts/business_hours.sh # every hour, 9am-5pm, weekdays only
@reboot /opt/mqtt_monitor/scripts/startup_check.sh # special string: run once at boot
```

Special strings: **@reboot**, **@daily** (=0 0 * * *), **@hourly**, **@weekly**, **@monthly**.

Critical cron gotchas (these bite everyone at least once)

```
# 1. cron runs with a MINIMAL environment – no PATH beyond /usr/bin:/bin, no interactive shell config loaded
# ALWAYS use absolute paths for both the script AND anything the script calls.

# WRONG (relies on PATH, working directory, python being found):
* * * * * python3 monitor.py

# RIGHT:
* * * * * /usr/bin/python3 /opt/mqtt_monitor/monitor.py

# 2. cron jobs don't inherit your shell's environment variables – set what you need explicitly in the script
# or in the crontab itself:
PATH=/usr/local/bin:/usr/bin:/bin
MQTT_BROKER=localhost
0 2 * * * /opt/mqtt_monitor/scripts/backup.sh

# 3. ALWAYS redirect output, or you'll silently lose errors (mail delivery is usually not configured):
0 2 * * * /opt/mqtt_monitor/scripts/backup.sh >> /var/log/mqtt_monitor/backup.log 2>&1
```

System-wide cron locations (alternative to per-user crontab)

```
/etc/crontab          # system crontab – includes a username field (unlike user crontabs)
/etc/cron.d/          # drop-in directory for package-managed or custom system cron jobs
/etc/cron.daily/      # scripts here run once daily (via run-parts, triggered by cron itself)
/etc/cron.hourly/     /etc/cron.weekly/ /etc/cron.monthly/ # same idea, different frequency
```

at — run something ONCE at a specific future time (not recurring)

```
echo "/opt/mqtt_monitor/scripts/one_time_task.sh" | at 23:00 # run once tonight at 11pm
at now + 30 minutes <<< "/opt/mqtt_monitor/scripts/reminder.sh" # run once, 30 minutes from now
atq # list pending 'at' jobs
atrm 3 # cancel job number 3
```


systemd timers — the modern alternative, with logging/dependency benefits

/etc/systemd/system/mqtt_backup.service:

```
[Unit]
Description=Backup MQTT monitor database

[Service]
Type=oneshot
ExecStart=/opt/mqtt_monitor/scripts/backup.sh
```

/etc/systemd/system/mqtt_backup.timer:

```
[Unit]
Description=Run MQTT backup daily at 2am

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true

[Install]
WantedBy=timers.target

sudo systemctl daemon-reload
sudo systemctl enable --now mqtt_backup.timer
systemctl list-timers                                # confirm it's scheduled, see next run time
journalctl -u mqtt_backup.service                    # logs from each run – this is the big advantage over cron
```

Persistent=true means: if the machine was off/asleep when the timer should have fired, run it once at next boot instead of silently skipping — very relevant for embedded devices that get power-cycled.

Key takeaways

- Cron's #1 failure mode is environment differences from your interactive shell — always use absolute paths and redirect output explicitly.
 - systemd timers give you **journalctl**-based logging and **Persistent=true** (catch-up after downtime) for free — prefer them for anything on a systemd-managed embedded device.
 - **atq/atrm** for genuine one-off scheduled tasks; don't misuse cron with a self-deleting entry for that.
-

DAY 23 — Logging & Log Management

Concept

Production systems generate logs continuously; managing them means knowing where they live, how to query them efficiently, and how to prevent them from consuming all your disk (especially critical on space-constrained embedded storage).

Traditional syslog-style logs

```
tail -F /var/log/syslog          # Debian/Ubuntu main system log
tail -F /var/log/messages       # RHEL/CentOS equivalent
tail -F /var/log/auth.log       # authentication/sudo/SSH login events
tail -F /var/log/kern.log       # kernel messages logged via syslog
ls /var/log/                    # survey what's being logged on this system
```

journalctl deep dive (systemd's structured logging)

```
journalctl                      # entire journal, oldest first, paged
journalctl -r                   # reverse: newest first
journalctl -k                   # kernel messages only (like dmesg, but persistent across reboots)
journalctl -u mqtt_monitor -u mosquitto # multiple units at once
journalctl --since "2026-07-20 09:00:00" --until "2026-07-20 10:00:00" # precise time window
journalctl -p warning           # priority filter: emerg/alert/crit/err/warning/notif-emerg
journalctl -p err -b            # errors and above, since last boot
journalctl -o json-pretty -u mqtt_monitor -n 5 # structured JSON output – useful for feeding in
journalctl --output=cat -u mqtt_monitor        # just the message text, no metadata prefix -
```

Persistent vs volatile journal storage

```
cat /etc/systemd/journald.conf | grep -i storage # check current setting
# Storage=volatile -> journal lives only in RAM (/run/log/journal), lost on reboot
# Storage=persistent -> journal written to disk (/var/log/journal), survives reboot
sudo mkdir -p /var/log/journal                  # creating this directory itself triggers persistent storage
sudo systemctl restart systemd-journald
```

logrotate — preventing traditional log files from growing forever

```
cat /etc/logrotate.conf          # global logrotate settings
ls /etc/logrotate.d/             # per-service config (mosquitto, nginx, etc. often drop a file here)
cat /etc/logrotate.d/mosquitto   # example: see how an existing service configures its own rotation
```

Custom logrotate config for your own app, `/etc/logrotate.d/mqtt_monitor`:

```
/var/log/mqtt_monitor/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 monitoruser monitoruser
}
```

- **daily** + **rotate 14** = keep 14 days of rotated logs
- **compress** = gzip old logs; **delaycompress** = don't compress the most recently rotated one yet (some apps still have it open briefly)
- **missingok** = don't error if the log file doesn't exist; **notifempty** = don't rotate an empty file
- **create 0640 user group** = after rotating, create a fresh log file with these permissions/ownership

```
sudo logrotate -d /etc/logrotate.d/mqtt_monitor      # DRY RUN – test the config without actually rotating
sudo logrotate -f /etc/logrotate.d/mqtt_monitor      # force rotation right now (for testing the real bel
```

Application-level logging habits (what your own scripts/services should do)

- Log to `stdout/stderr` from anything running under `systemd` (`StandardOutput=journal` in the unit file) — let `journald` handle storage/rotation, don't reinvent file-based logging inside the app itself unless you have a specific reason.
- If you DO log to files directly (e.g. a standalone script not under `systemd`), always pair it with a `logrotate` config — an unrotated log is a slow-motion disk-full incident waiting to happen, especially on an SD card.

Key takeaways

- `journalctl -u <service> --since ... --until ...` is your primary tool for post-incident investigation — precise time windowing beats scrolling through `less`.
 - Set `Storage=persistent` deliberately if you need logs to survive a reboot — the default on many minimal/embedded images is volatile (RAM-only).
 - Any log file not managed by `journald` needs a `logrotate` config, full stop — this is especially non-negotiable on flash storage with finite write endurance and limited capacity.
-

DAY 24 — Advanced find, grep/sed/awk Combinations

Concept

find is the tool for locating files by attribute (name, type, size, age, permissions) — and it composes powerfully with everything from Days 6-9. This is where individually-simple tools combine into genuinely powerful one-liners.

find fundamentals

```
find . -name "*.log"           # find by name pattern, current dir and below
find /var/log -iname "*.LOG"   # case-insensitive name match
find . -type f                 # files only
find . -type d                 # directories only
find . -type l                 # symlinks only

find . -mtime -1               # modified in the last 1 day
find . -mtime +30              # modified MORE than 30 days ago
find . -mmin -60               # modified in the last 60 minutes
find /var/log -name "*.log" -size +100M   # larger than 100MB
find /var/log -name "*.log" -size -1k     # smaller than 1KB

find . -perm 644               # exact permission match
find . -perm -u+x              # has at least owner-execute set

find / -user monitoruser 2>/dev/null     # owned by a specific user (suppress permission-denied noise)
find / -group dialout 2>/dev/null         # owned by a specific group

find . -empty                   # empty files or directories
```

find + action (the real power)

```
find . -name "*.tmp" -delete           # delete matches directly (built-in, no xargs needed)
find . -name "*.log" -mtime +30 -delete # delete logs older than 30 days
find . -name "*.py" -exec chmod 644 {} \; # run a command on each match; {} = the match
find . -name "*.sh" -exec chmod +x {} + # + batches multiple matches into fewer commands
find . -type f -name "*.log" -exec grep -l "ERROR" {} \; # find files, then filter by content
find /opt/mqtt_monitor -name "*.pyc" -exec rm {} \; # clean up compiled Python cache files
```

Practical combined pipelines

```
# Find all Python files modified in the last day that mention "mqtt"
find . -name "*.py" -mtime -1 | xargs grep -l "mqtt"

# Find and archive logs older than 30 days, then delete the originals
find /var/log/mqtt_monitor -name "*.log" -mtime +30 -print0 | \
  xargs -0 tar -czvf old_logs_$(date +%F).tar.gz --remove-files

# Top 10 largest files under a directory
find . -type f -exec du -h {} + | sort -rh | head -10

# Count occurrences of "ERROR" per log file, sorted by count descending
for f in /var/log/mqtt_monitor/*.log; do
  echo "$(grep -c ERROR "$f") $f"
done | sort -rn

# Extract unique IP addresses from an access log, sorted by frequency
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -20

# Replace a config value across every .conf file in a directory tree
find /etc/mqtt_monitor -name "*.conf" -exec sed -i 's/old_broker/new_broker/g' {} +

# Find all files NOT matching a pattern (useful for cleanup audits)
```

```
find . -type f ! -name "*.py" ! -name "*.md"
```

grep with find, the other direction: recursive grep is often simpler

```
grep -rl "mqtt_connect" --include="*.py" .          # grep's own recursive+filter flags often replace a find
grep -rn "TODO" --include="*.py" --exclude-dir=venv .  # exclude directories (e.g. virtualenvs) from a recursive search
```

Key takeaways

- `find ... -exec ... +` (batched) is more efficient than `-exec ... \;` (one invocation per file) — prefer `+` when the command supports multiple trailing arguments.
 - `find -print0 | xargs -0` is the safe combination for filenames with spaces/special characters — make this your default for anything programmatic.
 - Before reaching for `find | xargs grep`, check whether `grep -r --include=... --exclude-dir=...` alone already does the job — often simpler.
-

DAY 25 — Advanced Process & Resource Management

Concept

Beyond starting/stopping processes, production and embedded systems need resource control — priority tuning, limiting resource consumption, and tracing what a process is actually doing at the syscall level when something misbehaves.

Priority: *nice* / *renice*

```
nice -n 10 ./heavy_batch_job.sh      # start a process with LOWER priority (higher niceness = less CPU)
nice -n -5 ./realtime_sensitive_task.sh # HIGHER priority (negative niceness) – requires root/sudo for root
renice -n 5 -p 1234                  # change the niceness of an ALREADY-RUNNING process by PID
ps -o pid,ni,cmd -p 1234              # check a process's current niceness (NI column)
top                                    # niceness shown in the 'NI' column, alongside PR (actual priority)
```

Practical use: on a resource-constrained embedded board running both your `mqtt_monitor` service and periodic batch analysis, *nice* the batch job so it doesn't starve the monitor of CPU during bursts.

Resource limits: *ulimit*

```
ulimit -a      # show all current limits for this shell
ulimit -n      # max open file descriptors (very relevant for a service handling many MQTT)
ulimit -n 4096 # raise it for the CURRENT shell session (and anything it launches)
ulimit -u      # max processes/threads for this user
ulimit -v      # max virtual memory (KB)
```

For a systemd service, set limits in the unit file instead of the shell (this is the correct place for a production service):

```
[Service]
LimitNOFILE=8192
LimitNPROC=100
```

cgroups — where systemd's resource control actually comes from (conceptual + practical entry point)

```
systemd-cgtop      # live view of resource usage PER cgroup (per service/slice) – like top
systemctl status mqtt_monitor # shows the cgroup path for this service
cat /sys/fs/cgroup/system.slice/mqtt_monitor.service/memory.current # current memory usage of this specific cgroup
```

Setting resource limits on a systemd service via unit file (uses cgroups under the hood — no need to manipulate cgroup files by hand):

```
[Service]
MemoryMax=256M
CPUQuota=50%
```

This caps `mqtt_monitor` at 256MB RAM and 50% of one CPU core — genuinely useful on a resource-constrained Raspberry Pi shared with other services.

Tracing what a misbehaving process is doing

```
strace -p 1234      # attach to a running process, show every syscall it makes live (Ctrl+C to detach)
strace -c ./monitor.py # run and summarize syscall counts/time – quick "what's it spending time on"
strace -e trace=open,read,write ./monitor.py # filter to specific syscall types (e.g., just file I/O)
strace -f -p 1234    # follow child processes/threads too (-f)

ltrace ./some_binary # like strace, but traces LIBRARY calls instead of syscalls (less common)

lsof -p 1234          # list all open files (and sockets, since "everything is a file") held by process
```

```
lsof -i :1883 # what process is using TCP/UDP port 1883?
lsof /var/log/mqtt_monitor/app.log # what process(es) currently have this specific file open?
```

Practical debugging scenario: "why is this process hung / using too much memory / not releasing a file"

```
ps aux | grep mqtt_monitor # find the PID
cat /proc/1234/status # detailed memory/state info straight from the kernel's view
ls -la /proc/1234/fd/ # every file descriptor currently open by this process
strace -p 1234 # see what it's actually doing (or NOT doing, if it's blocked)
lsof -p 1234 | grep -i mosquitto # is it still connected to the broker, or has that socket been closed?
```

Key takeaways

- `nice/renice` for CPU scheduling priority; systemd unit directives (`MemoryMax`, `CPUQuota`, `LimitNOFILE`) for hard resource caps in production — prefer the unit file over ad-hoc `ulimit` for anything running as a service.
 - `strace -p <pid>` is your first move when a process is "hung" and you need to know if it's blocked on I/O, a lock, or genuinely spinning.
 - `lsof -i :<port>` and `lsof -p <pid>` answer "what's using this" and "what is this process touching" respectively — both come up constantly in real debugging.
-

DAY 26 — Advanced Networking: Firewalls, Packet Capture & netcat

Concept

Beyond basic connectivity checks (Day 13), production/embedded systems need firewall rules to restrict access, and packet-level inspection to debug protocol-level issues (like malformed MQTT traffic).

netcat (nc) — the network Swiss Army knife

```
nc -zv broker.local 1883          # -z = scan/zero-I/O mode (just test if port is open), -v = verbose
nc -zv broker.local 1880-1890     # scan a port range
nc -l -p 9000                     # listen on port 9000 (server side) – receives whatever's sent to it
nc broker.local 9000              # connect and send stdin to that listening port (client side)
echo "test message" | nc -q1 broker.local 9000 # send one message and close (-q1 = quit 1 second after send)
```

Practical: `nc -zv` is a fast way to confirm "is the firewall/network blocking this port" vs. "is the application itself not responding" — separates network-layer from application-layer problems.

tcpdump — packet capture at the command line

```
sudo tcpdump -i eth0              # capture on interface eth0, print summary of each packet live
sudo tcpdump -i any               # capture on all interfaces
sudo tcpdump -i eth0 port 1883    # filter: only MQTT broker port traffic
sudo tcpdump -i eth0 host 192.168.1.50 # filter: only traffic to/from a specific host
sudo tcpdump -i eth0 -w capture.pcap # write raw capture to a file (for later analysis, e.g. Wireshark)
sudo tcpdump -r capture.pcap      # read back a previously saved capture
sudo tcpdump -i eth0 -A port 1883 # -A = print packet content in ASCII (useful for troubleshooting)
sudo tcpdump -i eth0 -X port 1883 # -X = print both hex AND ASCII (useful for inspection)
sudo tcpdump -i eth0 -c 20 port 1883 # capture exactly 20 packets then stop
```

Practical use case directly relevant to your stack: confirming whether your embedded device is actually sending MQTT CONNECT packets, or whether the connection is failing before the application layer even engages.

Basic firewall management (ufw — the friendly front-end most relevant on Debian/Ubuntu/RPi)

```
sudo ufw status                   # current firewall state and rules
sudo ufw enable                   # turn the firewall on
sudo ufw allow 1883/tcp           # allow MQTT broker port
sudo ufw allow from 192.168.1.0/24 to any port 1883 # allow only a specific subnet to reach that port
sudo ufw allow ssh                # explicitly ensure SSH stays allowed (do this BEFORE enabling ufw)
sudo ufw deny 23                  # explicitly deny telnet
sudo ufw delete allow 1883/tcp    # remove a specific rule
sudo ufw status numbered         # show rules with index numbers, for targeted deletion
sudo ufw delete 3                # delete rule #3 from the numbered list
```

Critical safety habit for remote/headless devices: **always** `ufw allow ssh` (or your actual SSH port) **before** `ufw enable` — enabling the firewall without an explicit allow rule for your remote access method can permanently lock you out of a headless embedded device.

iptables/nftables (lower-level, what ufw configures under the hood)

```
sudo iptables -L -v -n           # list current rules, verbose, numeric (no DNS lookups slowing it down)
sudo iptables -L -v -n --line-numbers # with rule numbers, for targeted deletion

sudo nft list ruleset            # nftables equivalent (the modern replacement iptables is transitioning to)
```


In practice: use `ufw` for day-to-day rule management; understand that it's writing `iptables/nftables` rules underneath, which is useful context when troubleshooting a rule that "should" be working per `ufw status` but isn't behaving as expected.

Key takeaways

- `nc -zv` isolates network/firewall problems from application problems — always check this before assuming your MQTT client/broker config is wrong.
 - `tcpdump -i <iface> port <port> -A` gets you real packet content for a text-based protocol without needing Wireshark.
 - **Always allow SSH before enabling ufw** on any remote/headless device — this single ordering mistake is how people lock themselves out of Raspberry Pis in the field.
-

DAY 27 — Git as a Command-Line Tool

Concept

Git is technically separate from "the shell," but in practice it's inseparable from real command-line workflows — every project you build (including `mqtt_monitor`) lives in it. This day covers the CLI fluency layer, not git internals.

Daily-driver commands

```
git init                                # initialize a new repo in the current directory
git clone git@github.com:user/repo.git  # clone an existing remote repo
git status                              # what's changed, what's staged, what's untracked – run this constantly
git diff                                # unstaged changes, line-by-line
git diff --staged                       # staged changes, line-by-line
git add file.py                         # stage a specific file
git add .                               # stage everything changed/new in current dir and below
git add -p                              # stage interactively, hunk by hunk – review before committing
git commit -m "Add serial reconnect logic" # commit staged changes with a message
git commit -am "message"                # stage all tracked-file changes AND commit in one step
git log                                 # commit history
git log --oneline --graph --all         # compact, visual history across all branches
```

Branching

```
git branch                             # list local branches
git branch feature/reconnect-logic     # create a new branch
git checkout feature/reconnect-logic   # switch to it
git checkout -b feature/reconnect-logic # create AND switch in one step
git switch feature/reconnect-logic     # modern equivalent of checkout for switching branches
git switch -c feature/reconnect-logic  # modern equivalent of checkout -b

git merge feature/reconnect-logic      # merge that branch into the current branch
git branch -d feature/reconnect-logic  # delete a branch (only if already merged)
git branch -D feature/reconnect-logic  # force delete (even if unmerged – be sure)
```

Remotes

```
git remote -v                          # list configured remotes and their URLs
git remote add origin git@github.com:user/repo.git
git push origin main                   # push local commits to remote
git push -u origin main                # push AND set upstream tracking (so plain `git push` works afterwards)
git pull                               # fetch + merge from tracked remote branch
git fetch                              # fetch remote changes WITHOUT merging (safer to inspect first)
```

Undoing things (the part people actually search for under pressure)

```
git restore file.py                    # discard unstaged changes to a file (git 2.23+, replaces 'checkout')
git restore --staged file.py           # unstage a file, keep the changes in the working directory
git reset --soft HEAD~1                # undo last commit, keep changes staged
git reset --mixed HEAD~1               # undo last commit, keep changes unstaged (default mode)
git reset --hard HEAD~1                # undo last commit AND discard the changes entirely – destructive
git revert <commit_hash>               # create a NEW commit that undoes a previous one – safe to use
git stash                              # shelve current uncommitted changes temporarily
git stash pop                          # reapply the most recently stashed changes
git stash list                         # see all stashed sets
```

Rule of thumb: `reset --hard` rewrites history and is fine on commits you haven't pushed; once pushed/shared, use `revert` instead, which adds a new commit rather than rewriting what others may have already pulled.

Inspecting history

```
git log -p file.py          # full diff history for one specific file
git blame file.py           # who last changed each line, and in which commit
git show <commit_hash>      # full diff introduced by a specific commit
git diff main..feature/reconnect-logic  # diff between two branches
```

.gitignore essentials for a Python/C++ project

```
__pycache__/  
*.pyc  
.venv/  
build/  
*.o  
*.db  
.env
```

Key takeaways

- **git status** before and after nearly every action — it tells you exactly what state you're in.
 - **git switch/git restore** are the modern, clearer-intent replacements for overloaded **git checkout** — prefer them going forward.
 - **revert** (safe, additive) vs **reset --hard** (destructive, history-rewriting) — the deciding factor is whether the commit has already been pushed/shared.
-

DAY 28 — Security & Hardening

Concept

Command-line security hardening for a device you actually deploy (especially an internet-reachable or field-deployed embedded box) — the practical baseline, not a compliance checklist.

SSH hardening

`/etc/ssh/sshd_config` key settings:

```
PermitRootLogin no                # never allow direct root SSH login
PasswordAuthentication no         # force key-based auth only, once your keys are confirmed working
PubkeyAuthentication yes
Port 2222                        # optional: move off the default port to cut down on automated scans
AllowUsers george monitoruser    # explicit allowlist of who may SSH in at all
MaxAuthTries 3                   # limit auth attempts before disconnecting
ClientAliveInterval 300          # disconnect idle sessions after a timeout

sudo sshd -t                     # TEST the config for syntax errors BEFORE restarting – a broken config
sudo systemctl restart sshd
```

Critical: always test config syntax and keep an existing SSH session open while restarting `sshd`, in case the new config breaks something — don't close your only connection until you've confirmed a fresh connection works.

fail2ban — automatic banning of brute-force attempts

```
sudo apt install fail2ban
sudo systemctl enable --now fail2ban
sudo fail2ban-client status      # list active jails
sudo fail2ban-client status sshd # details for the sshd jail specifically: banned IPs, at
sudo fail2ban-client set sshd unbanip 1.2.3.4 # manually unban an IP
```

`/etc/fail2ban/jail.local` (create this rather than editing `jail.conf` directly — it overrides cleanly and survives package updates):

```
[sshd]
enabled = true
maxretry = 3
bantime = 3600
findtime = 600
```

File integrity monitoring

```
sudo apt install aide            # Advanced Intrusion Detection Environment
sudo aideinit                    # build the initial baseline database
sudo aide --check                 # compare current filesystem state against the baseline, report changes
```

auditd — kernel-level audit logging (who did what, at the syscall level)

```
sudo apt install auditd
sudo systemctl enable --now auditd
sudo auditctl -w /etc/passwd -p wa -k passwd_changes # watch a file for writes/attribute changes, tag with key
sudo ausearch -k passwd_changes                      # search audit log by that key
sudo auditctl -l                                     # list current audit rules
```

General hardening checklist (command-line actionable)

```
sudo passwd -l some_unused_account # lock an account (disable password login) without deleting it
sudo chage -l george                # check password aging/expiry policy for a user
sudo chmod 600 ~/.ssh/id_ed25519    # private keys must never be group/world readable
```

```
sudo find / -perm -4000 -type f 2>/dev/null    # audit: find all setuid binaries on the system (potential p
sudo find / -nouser -o -nogroup 2>/dev/null    # find files owned by a UID/GID that no longer maps to a
last                                           # show recent login history
lastb                                         # show recent FAILED login attempts (may need sudo,
who                                           # who's currently logged in
w                                             # who's logged in + what they're running
```

Key takeaways

- `PermitRootLogin no` + `PasswordAuthentication no` + `fail2ban` is the practical baseline for any internet-facing or field-deployed SSH endpoint.
 - Always `sshd -t` before restarting SSH, and keep your current session open until you've verified a new connection works.
 - `find / -perm -4000` (setuid audit) and `lastb` (failed logins) are quick, high-value periodic checks for a deployed device.
-

DAY 29 — Performance Monitoring & Troubleshooting

Concept

When something is slow, hung, or resource-starved, there's a standard diagnostic sequence: CPU, memory, disk I/O, then application-specific state. Command-line tools cover every layer.

CPU

```
top                # live overview; look at %CPU per process and load average
mpstat 1           # per-core CPU breakdown, refreshed every 1 second (apt install sysstat)
vmstat 1           # system-wide CPU/memory/I/O summary, refreshed every 1 second – 'r' column =
uptime             # load average: 1/5/15-minute averages; compare against nproc to gauge saturation
nproc              # number of cores, for context on whether a load average is actually high
```

Reading load average: a load average of 4.0 on a 4-core machine means fully utilized; on a 1-core Raspberry Pi Zero, a load average of 4.0 means things are queuing up badly.

Memory

```
free -h            # total/used/free/available memory and swap, human-readable
vmstat 1           # 'si'/'so' columns = swap in/out – sustained non-zero values mean you're swapping
cat /proc/meminfo   # full detailed breakdown straight from the kernel
ps aux --sort=-%mem | head -10 # top 10 processes by memory usage
smem -t            # more accurate memory reporting, accounts for shared memory properly
```

Disk I/O

```
iostat -x 1        # extended disk I/O stats, refreshed every 1s (apt install sysstat) – watch %util
iotop              # like top, but for disk I/O per-process (apt install iotop, needs sudo)
df -h              # is a filesystem actually full? (separate issue from I/O speed)
```

Combined system snapshot

```
sar -u 1 5         # sar: historical + live system activity reporting, CPU usage over 5 samples, 1s
sar -r 1 5         # memory usage samples
sar -b 1 5         # I/O transfer rate samples
htop               # single best "give me the overview right now" tool – CPU per-core, memory, disk
```

Application-specific troubleshooting sequence (practical, in order)

```
# 1. Is the service even running, and what does it think happened?
systemctl status mqtt_monitor
journalctl -u mqtt_monitor -n 100 --no-pager

# 2. Is it consuming abnormal resources?
ps aux | grep mqtt_monitor
top -p $(pgrep -d',' -f mqtt_monitor)

# 3. Is it stuck on I/O, a lock, or a syscall?
strace -p $(pgrep -f mqtt_monitor) -c      # Ctrl+C after a few seconds, review the syscall summary

# 4. Is it a network-layer problem underneath?
ss -tnp | grep $(pgrep -f mqtt_monitor)
nc -zv broker.local 1883

# 5. Is the underlying storage/filesystem healthy?
df -h
dmesg | grep -i -E "error|fail" | tail -20
```

Boot-time troubleshooting

```
systemd-analyze           # total boot time breakdown
systemd-analyze blame      # which services took longest to start, sorted
systemd-analyze critical-chain # the critical dependency chain that determined total boot time
journalctl -b -p err       # errors from the current boot only
journalctl -b -1           # full log from the PREVIOUS boot (useful after an unexpected
```

Key takeaways

- Diagnostic order under pressure: CPU (**top/vmstat**) → memory (**free/vmstat**) → disk I/O (**iostat/iotop**) → application state (**journalctl/strace**) → network (**ss/nc**) — work outward from the system layer to the application layer.
 - Load average is meaningless without knowing core count (**nproc**) — always read them together.
 - **systemd-analyze blame** and **journalctl -b -1** are the first two commands after "the device rebooted unexpectedly and I don't know why."
-

DAY 30 — Capstone: Building a Field Diagnostics & Monitoring Toolkit

Concept

This day integrates Days 1-29 into one coherent, deployable toolkit for exactly the kind of system you build in practice: a headless embedded Linux device running `mqtt_monitor`, accessed remotely, that needs to be observable, recoverable, and secure without physical access.

Toolkit component 1: Health check script (Days 19-21, 25, 29)

```
#!/usr/bin/env bash
# /opt/mqtt_monitor/scripts/health_check.sh
set -euo pipefail

LOGFILE="/var/log/mqtt_monitor/health_check.log"
SERVICE="mqtt_monitor"
BROKER_HOST="localhost"
BROKER_PORT="1883"
DEVICE_PATHS=("/dev/sensor_gateway" "/dev/ttyUSB0" "/dev/ttyACM0")

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $" | tee -a "$LOGFILE"
}

check_service() {
    if systemctl is-active --quiet "$SERVICE"; then
        log "OK: $SERVICE is active"
        return 0
    else
        log "FAIL: $SERVICE is NOT active – attempting restart"
        systemctl restart "$SERVICE"
        sleep 3
        if systemctl is-active --quiet "$SERVICE"; then
            log "RECOVERED: $SERVICE restarted successfully"
        else
            log "CRITICAL: $SERVICE failed to restart"
            return 1
        fi
    fi
}

check_broker_reachable() {
    if nc -z -w2 "$BROKER_HOST" "$BROKER_PORT"; then
        log "OK: broker reachable at $BROKER_HOST:$BROKER_PORT"
    else
        log "FAIL: broker unreachable at $BROKER_HOST:$BROKER_PORT"
        return 1
    fi
}

check_serial_device() {
    for device in "${DEVICE_PATHS[@]}; do
        if [ -e "$device" ]; then
            log "OK: serial device present at $device"
            return 0
        fi
    done
    log "FAIL: no serial device found among: ${DEVICE_PATHS[*]}"
    return 1
}
```



```

check_disk_space() {
    local usage
    usage=$(df -h / | awk 'NR==2 {print $5}' | tr -d '%')
    if [ "$usage" -ge 90 ]; then
        log "WARNING: root filesystem at ${usage}% capacity"
        return 1
    else
        log "OK: root filesystem at ${usage}% capacity"
    fi
}

main() {
    local exit_code=0
    check_service || exit_code=1
    check_broker_reachable || exit_code=1
    check_serial_device || exit_code=1
    check_disk_space || exit_code=1
    exit "$exit_code"
}

main

```

Toolkit component 2: systemd timer for the health check (Days 12, 22)

/etc/systemd/system/mqtt_health_check.service:

```

[Unit]
Description=MQTT Monitor health check

[Service]
Type=oneshot
ExecStart=/opt/mqtt_monitor/scripts/health_check.sh

```

/etc/systemd/system/mqtt_health_check.timer:

```

[Unit]
Description=Run health check every 5 minutes

[Timer]
OnCalendar=*:0/5
Persistent=true

[Install]
WantedBy=timers.target

```

Toolkit component 3: Remote diagnostics one-liner (Days 13, 14, 23)

Run from your workstation against a fleet of devices listed in *~/.ssh/config*:

```

#!/usr/bin/env bash
# fleet_diag.sh – quick remote health snapshot across multiple devices
set -euo pipefail

DEVICES=("rpi-site1" "rpi-site2" "beaglebone-site3") # matches Host aliases in ~/.ssh/config

for host in "${DEVICES[@]}; do
    echo "=== $host ==="
    ssh -o ConnectTimeout=5 "$host" '
        systemctl is-active mqtt_monitor;
        df -h / | tail -1;
        journalctl -u mqtt_monitor -p err --since "1 hour ago" --no-pager | tail -5
    ' || echo "UNREACHABLE: $host"
    echo
done

```

Toolkit component 4: Log aggregation & backup (Days 14, 16, 23)

```
#!/usr/bin/env bash
# pull_logs.sh – pull journal logs and app logs from a remote device for offline analysis
set -euo pipefail

HOST="$1"
DEST="./diagnostics_$(date +%Y%m%d_%H%M%S)_${HOST}"
mkdir -p "$DEST"

ssh "$HOST" 'journalctl -u mqtt_monitor --since "24 hours ago" --no-pager' > "$DEST/mqtt_monitor_journal.l
rsync -avz "$HOST:/var/log/mqtt_monitor/" "$DEST/app_logs/"
ssh "$HOST" 'df -h; free -h; uptime; systemctl status mqtt_monitor --no-pager' > "$DEST/system_snapshot.tx

tar -czvf "${DEST}.tar.gz" "$DEST"
echo "Diagnostics bundle: ${DEST}.tar.gz"
```

Toolkit component 5: Security baseline application (Day 28)

```
#!/usr/bin/env bash
# harden_device.sh – apply the baseline hardening to a freshly provisioned device
set -euo pipefail

echo "Configuring firewall..."
ufw allow ssh
ufw allow 1883/tcp
ufw --force enable

echo "Installing fail2ban..."
apt install -y fail2ban
systemctl enable --now fail2ban

echo "Hardening SSH..."
sed -i 's/^#*PermitRootLogin.*PermitRootLogin no/' /etc/ssh/sshd_config
sed -i 's/^#*PasswordAuthentication.*PasswordAuthentication no/' /etc/ssh/sshd_config
sshd -t && systemctl restart sshd

echo "Setting up log rotation..."
cat > /etc/logrotate.d/mqtt_monitor << 'EOF'
/var/log/mqtt_monitor/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
}
EOF

echo "Hardening complete."
```

How the pieces fit together

Device boots

- > systemd starts mqtt_monitor.service (Day 12)
- > mqtt_health_check.timer fires every 5 min (Day 22), checks service/broker/device/disk (Days 25, 13, 19)
- > journald captures everything, rotated/bounded (Day 23)
- > ufw + fail2ban + hardened sshd protect the exposed surface (Day 28)
- > fleet_diag.sh from your workstation gives a fleet-wide status check over SSH (Days 13, 14)
- > pull_logs.sh grabs a full diagnostics bundle when something needs deeper investigation (Day 16, 24, 28)

Key takeaways

- Production reliability on an embedded device comes from composing simple, individually-understood pieces (a script, a timer, a unit file, a firewall rule) — not from one clever tool.
- Every script above assumes `set -euo pipefail`, absolute paths, explicit logging, and quoted variables — the habits from Days 19-21 are what make automation trustworthy enough to run unattended on a device you can't walk up to.
- This toolkit structure — health check + scheduled timer + remote diagnostics + log pull + security baseline — generalizes directly to any headless Linux deployment, not just MQTT-based monitoring.

End of curriculum. Total: 30 days, command-line fundamentals through production embedded-Linux operational practice.